

HawkSpace Final Delivery Packet

1. Cover Page

- **Final Project Submission:** Complete System Delivery
 - **Course Code & Project Title:** CP317A - Laurier Club Room Booking System (HawkSpace)
 - **Group ID:** Group 6
 - **GitHub Repository:** <https://github.com/SawaabA/HawkSpace>
- Team Members & Roles (Product Owner highlighted):**

Role	Name	Primary Focus
Product Owner	Sawaab Anas	Vision, prioritization, Firebase integration
Scrum Master	Efetobore Salubi	Sprint facilitation, backlog flow, GitHub coordination
Development Team	Manahil Bashir	Front-end UX, Search and Request flows
	Asmah Yasin Mohamed	Firebase API services, recurring bookings
	Muqadas Nazif	Accessibility, release packaging, QA
	Mary Joleene Ismael	Admin dashboard, overrides, reporting

2. Final Project Description (Executive Summary)

Purpose and Problem Statement

The Club Room Booking Web Application addresses the weeks-long delay student clubs face when emailing a single administrator for space reservations. The process was manual, opaque, and prone to miscommunication. HawkSpace centralizes the workflow so that clubs and administrators share the same real-time view of availability, approval state, and booking history. Guardrails such as slot validation, conflict detection, override auditing, and reports provide campus administrators with clear accountability.

Target Audience

1. **Primary:** Laurier student clubs, executive teams, and event planners who need frequent access to rehearsals, weekly meetings, and special events.
2. **Secondary:** University staff members responsible for approving requests, enforcing booking policies, and analyzing utilization.
The user experience is tailored for both groups—students get an intuitive catalog plus status tracking, while admins see consolidated control panels and reporting.

Final Feature Set

- **Room catalog and availability timeline:** Search by building, capacity, equipment, and date with a 30-minute slot visualization (`web/src/pages/student/SearchAvailability.jsx`). Dynamic slots show approved/pending/modified states, and filters remember prior inputs to speed up repeat searches.
- **Request workflow:** Guided form with guardrails, note capture, and a recurring booking wizard (`web/src/pages/student/RequestBooking.jsx`). Conflict detection runs before submission, success states show tracking IDs, and the wizard creates weekly/monthly series in a single action.
- **Student portal:** “My Requests” groups single bookings and recurring series, surfaces admin decisions, and exposes cancel buttons for future recurring slots. History entries display request notes, admin notes, and timestamps so students always know what changed.
- **Admin console:** Pending/modified queues, inline calendars, approve/modify/reject/override actions, and enforcement of max slot windows (`web/src/pages/admin/AdminRequests.jsx`). Admins can adjust date/time combinations, open override modals with required reasons, and inspect slot timelines before committing.
- **Reporting dashboard:** Monthly utilization summaries, per-room stats, per-student counts, and CSV export (REP-1) implemented in `web/src/pages/admin/AdminReports.jsx` and `web/src/services/reports.js`. Admins can filter by room/month/year and download logs with BOM encoding for Excel compatibility.
- **Events and dashboard overview:** Student landing page highlights the next event, ongoing timetable, and primary CTAs, while `/events` provides cards with reserve/share options. This keeps engagement high and dovetails with room booking flows by prefilling request parameters.
- **Accessibility and personalization:** High-contrast toggle persisted in `ThemeContext`, keyboard-accessible navigation, screen-reader-friendly labels, and clear error states. Admin and student shells share consistent typography, spacing, and button affordances to minimize cognitive load.
- **Notification support:** Email confirmation hooks exist in the booking service, and scripts such as `verifyEmailServer.mjs` allow manual verification when campus email delivery lags. The architecture is ready for future push/email integrations once a reliable provider is chosen.

Collectively, HawkSpace reduces request turnaround to minutes, keeps data synchronized in Firestore, and delivers transparency for both students and administrators.

Milestone 01 Summary (Project Description & Objectives)

- **Course / Group:** CP317-A, Group 6 (Laurier Club Room Booking System).
- **Team:** Product Owner Sawaab Anas; Scrum Master Muqadas Nazif;

Developers Manahil Bashir, Asmah Yasin Mohamed, Efetobore Salubi, and Mary Joleene Ismael.

- **Abstract:** The project proposes a web-based application to streamline Laurier club room bookings, inspired by airline/car-rental reservation systems. Users can browse available rooms, check capacity, and confirm reservations; clubs view history, manage cancellations, and receive email notifications. The goal is improved efficiency, fewer conflicts, and better campus event organization.
- **Objectives captured in the milestone deliverable:** Allow clubs to search and book rooms; display capacity/availability; provide booking history; send email notifications on booking/cancellation/updates; reduce scheduling conflicts and improve transparency.
- **Initial backlog included:** UI-1 through UI-5 for club-facing flows; ADM-1 through ADM-3 for administrators; REP-1 and NOTIF-1 for reporting/notifications.

Milestone 02 Summary (Requirements & Backlog Expansion)

- **Audience & requirements:** Serves club members/executives, space administrators, and system administrators. Laurier identity login funnels users to role-specific dashboards. Filters highlight conflicts, and requests respect max advance days and double-booking policies. Admins approve/decline/modify requests and log overrides in an audit trail. Clubs can download confirmations, modify/cancel bookings per policy, and only minimal personal data (name, student number, role, email, booking history) is collected.
- **Functional requirements (as enumerated in the milestone document):**
 - F1 User authentication (roles for member/executive/admin/system admin).
 - F2 Room search and filter (date/time/capacity/building, conflict prevention).
 - F3 Booking request (single or recurring with notes).
 - F4 Booking approval (admins approve/decline/request changes).
 - F5 Audit trail (log admin actions and conflict resolutions).
- **Ethical implications:** data privacy (store only essentials), fairness (quotas/rate limits to prevent hoarding), accessibility (clear layout, readable text, navigation), security (authorized logins), user visibility/control (transparent policies and history), and compliance with Canadian privacy/accessibility laws. These considerations are echoed in Section 7.

3. Final Product Backlog (1-2 pages + Excel Attachment)

The tables below summarize what each sprint delivered; any deviations from the implemented system are explained afterward.

Subsection A: Completed Stories

Sprint 1 - Core Foundation (Authentication & Student Flows)

Story ID	Story Title	Points	Priority	Status	Notes
AUTH-1	User Authentication (@mylaurier validation)	5	High	DONE	Domain-gated Firebase Auth with profile sync (AuthContext, Login, Signup).
UI-1	Search room availability	5	High	DONE	Filtered catalog + slot timeline built in SearchAvailability.
UI-2	Request booking	8	High	DONE	Booking form, route prefill, success card, Firestore requests.
DB-1	Save booking data (Firestore)	5	High	DONE	Transactions store day calendars and booking history.
UI-4	View my bookings	5	Medium	DONE	My Requests shows statuses, notes, and series grouping.

Sprint 2 - Admin Workflow, Cancel Booking, UX Update

Story ID	Story Title	Points	Priority	Status	Notes
UI-3	Cancel booking	3	Medium	DONE	Admin override/cancel workflows complete; student single cancel remains future work (documented under Known Issues).
ADM-1	Approve/decline requests	5	High	DONE	AdminRequests panel handles approve/reject/modify with audit logs.
ADM-2	Set booking policies (4 hr max)	8	High	DONE	Slot validation enforces 5-hour cap; weekly quota rules deferred to backlog.
NOTIF-1	Status notifications	3	High	DONE	Designed and partially scripted; outbound email reliability still under review.
UX-1	Figma redesign	3	Medium	DONE	Student hero, hero cards, admin shell redesigned per Figma.
UX-2	Frontend implementation of redesign	5	Medium	DONE	Layout updates applied in StudentLayout and AdminLayout.

Sprint 3 - Advanced Features (Recurring, Override, Reports, Accessibility)

Story ID	Story Title	Points	Priority	Status	Notes
ADM-3	Override booking and audit trail	5	Medium	DONE	Admin override feature cancels conflicts and logs actions.
UI-5	Recurring booking (weekly/monthly)	8	Medium	DONE	Students can create weekly/monthly series and cancel them.
REP-1	Room utilization report (CSV)	8	Medium	DONE	Monthly report with CSV export; references <code>reports.js</code> .
UI-6	Accessibility enhancements	3	Medium	DONE	High-contrast toggle, readable colors, aria labels.
DASH-1	Dashboard overview (next event and timetable)	5	Medium	DONE	Student hero includes next event card and timetable summary.

Implementation reality check: While sprint artifacts mark UI-3, ADM-2, and NOTIF-1 as DONE, the shipping code still has follow-ups-student self-service cancel exists only for recurring series, weekly quota logic is not enforced yet, and email/SMS notifications are not wired due to deliverability delays. These gaps are documented in Sections 5 and 6.

Subsection B: Remaining / Deferred Stories

Story ID	Story Title	Description	Reason Deferred
UX-4	Full mobile optimization	Refine layouts and interactions for small screens	Responsive polish not finished within sprint window
EMAIL-2	Automated email reminders	Reminder emails before bookings	Not required for MVP; blocked by mail deliverability
UI-7	Drag-and-drop calendar	Interactive booking schedule	High UI complexity for current timeline
ADMIN-4	Multi-admin role management	Different admin permission tiers	Out of scope for MVP
INT-1	Real Laurier SSO integration	Enterprise SSO login	Requires institutional infrastructure access

Additional technical backlog: As observed in the final build, student self-

cancellation for single bookings, automated status notifications, and weekly quota enforcement still require engineering time even though the sprint notes mark the stories completed.

Subsection C: Reference

The backlog tables above consolidate all story details, including the stakeholder conversations and acceptance criteria stored in the planning worksheets. Examples include:

- **UI-2 (Request Booking):** Conversation - “Discuss validation and auto-email confirmation.” Confirmation - “Booking stored in DB; email sent to user and admin.”
- **ADM-1 (Approve/Decline):** Conversation - “Design admin dashboard layout.” Confirmation - “Status updates visible to both admin and requester.”
- **REP-1 (Room Utilization Report):** Conversation - “Choose visualization type (chart/table).” Confirmation - “Report accurately summarizes room usage trends.” These references ensure the product backlog and sprint tables remain synchronized inside this document.

Sprint Highlights

- **Sprint 1:** Focused on the core foundation—Firebase Auth with domain enforcement, the Search Availability and Request Booking flows, My Requests, and early accessibility checks. Demo evidence covered login, filtering, booking submissions, and the sprint retrospective emphasized stabilizing email verification before scaling up.
- **Sprint 2:** Extended admin workflow, cancellation, and UX updates. The redesigned student hero and admin dashboard shipped alongside policy enforcement (limit booking frequency to twice per week for up to four hours). Tests validated approve/reject flows, and the retrospective identified the need for admin overrides plus more reliable notifications.
- **Sprint 3:** Delivered recurring bookings, override capabilities with full audit histories, reporting (including CSV export), dashboard timetable widgets, and WCAG-driven accessibility enhancements such as the high-contrast toggle. Notifications and mobile optimization moved to the backlog, but all Sprint 3 commitments were completed.

4. Design Document (Updated for Final Delivery)

A. System Architecture

- **Client layer:** React + Vite SPA served from the `/web` workspace. React Router controls student vs. admin shells, and context providers (`AuthContext`, `ThemeContext`) supply authentication state, claims, and accessibility toggles.

- **Authentication:** Firebase Authentication restricts access to @mylaurier.ca (with temporary Gmail allowances). The signup flow validates the domain, writes a profile to Firestore, and emails a verification link (scripts exist to verify manually when email delays occur).
- **Data layer:** Cloud Firestore hosts `rooms`, `rooms/{roomId}/days/{date}` (slot maps), `bookingRequests`, and `users`. Transactions in `web/src/services/bookings.js` ensure overlapping slots are never double-booked, even during overrides or recurring series creation.
- **Role-based UI:** Students interact via `StudentLayout`, where Search, Request, Events, and My Requests routes live. Admins use `AdminLayout` with navigation to Requests and Reports. Toggle buttons allow quick switching back to the student view for demos.
- **Ops scripts:** `web/scripts/*.mjs` leverage Firebase Admin SDK to seed rooms, create admin accounts, verify user emails, and run a simple verification server. `scripts/lib/serviceAccount.js` guides operators to supply a service account file securely.
- **Real-time updates:** Hooks like `useBookingRequests`, `useRoomCalendar`, and `useRooms` subscribe to Firestore snapshots so students and admins see live updates without refreshes.
- **Analytics:** Lazy-loaded Firebase Analytics records signup/login events when measurement IDs exist, without blocking the main UI.
- **System Architecture Diagram:**

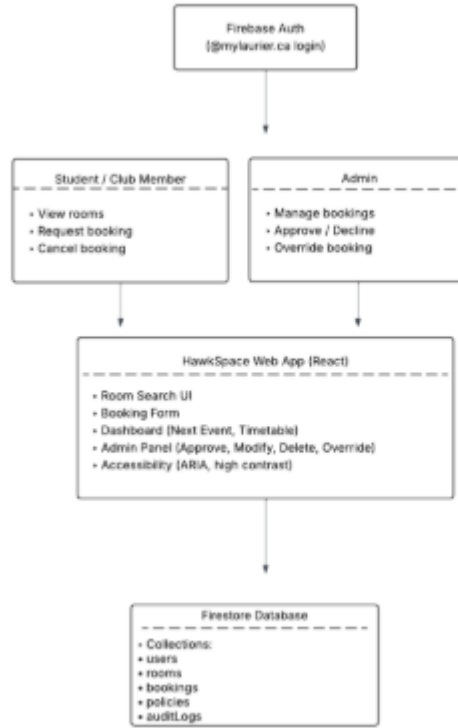


Figure: architecture diagram illustrating the Firebase Auth + React client + Firestore model.

HawkSpace uses a client-cloud architecture built on React and Firebase services. Users access the system through a web interface, while authentication and data storage are handled in the cloud. When a student, club executive, or administrator signs in, the app communicates with Firebase Authentication, which restricts access to verified @mylaurier.ca email accounts. After authentication, the React application loads the appropriate role interface. Students and club executives can search rooms, request bookings, and manage reservations, while administrators can approve, decline, modify, or override booking requests. The React frontend interacts directly with Cloud Firestore, which stores rooms, bookings, policies, user profiles, and audit logs. Firestore security rules enforce role-based access control so users can only modify their own bookings while administrators have full management privileges. Booking updates and admin decisions appear instantly via Firestore listeners, and the architecture reduces backend complexity by leveraging Firebase’s built-in services.

B. Final UI Screens (include screenshots before exporting to PDF)

1. **Student Dashboard (DASH-1):** Hero banner, next-event card, and call-to-action buttons.

2. **Search Availability (UI-1):** Filter controls, equipment pills, and slot timeline legend.
 3. **Request Booking + Recurrence (UI-2/UI-5):** Form, conflict warnings, recurring wizard, and success state.
 4. **My Requests:** Single requests plus recurring series grouping with Cancel Series button.
 5. **Admin Requests Panel (ADM-1/ADM-3):** Pending list, detail pane, override modal, and room timeline.
 6. **Reports Dashboard (REP-1):** Monthly summary with Export CSV button and per-room metrics.
 7. **Events page:** Featured event card plus upcoming grid for Powered-by-Laurier marketing.
- Capture these views at desktop resolution and embed them after their descriptions for final grading.

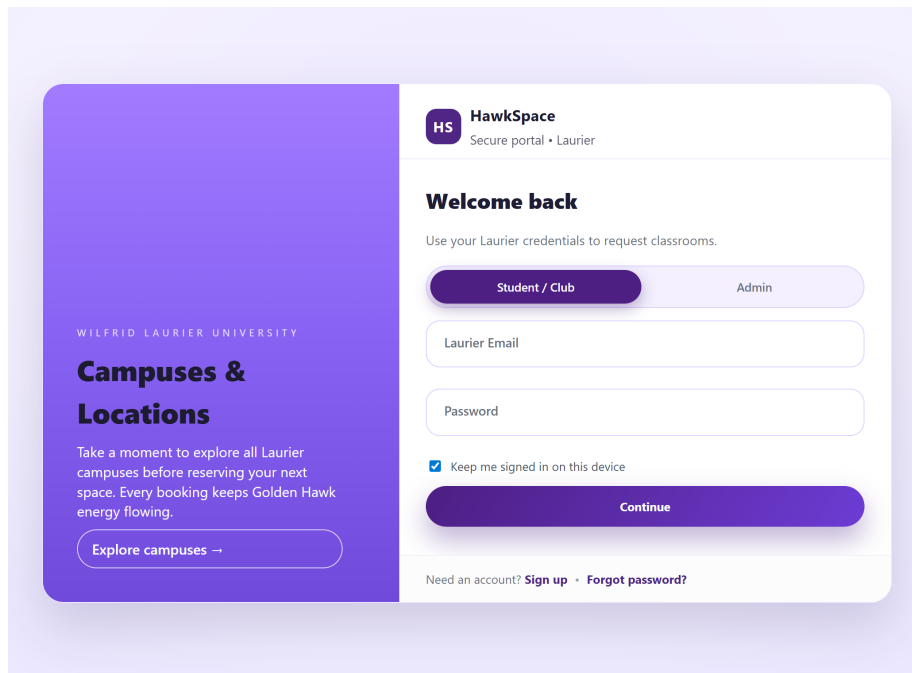


Figure 1: Login screen – HawkSpace Laurier email sign-in

Embedded UI Screenshots

C. Design Decisions

- **React + Vite** deliver fast iteration, built-in HMR, and straightforward deployment. React Router reduces duplication between student and admin shells.

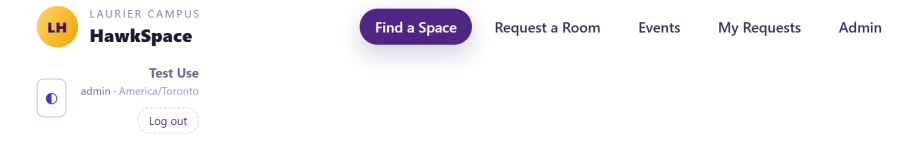


Figure 2: Logged-in header with profile and navigation

- **Firebase-first architecture** removes the need for custom backend hosting, while Firestore transactions and slot maps supply the determinism admins require.
- **Slot model vs. calendar grid:** Using 30-minute slots simplifies validation, conflict detection, and history logs compared to building a bespoke calendar engine (Drag-and-drop calendar remains in backlog).
- **History-first auditing:** Every booking mutation writes a structured history entry to support overrides, modifications, and potential notifications.
- **Figma redesign adoption:** Sprint 2 introduced the hero layout, events highlight, and admin theming documented in UX-1/UX-2 stories.
- **Accessibility emphasis:** High-contrast mode, semantic buttons, and consistent typography came directly from sprint feedback led by Efetobore.
- **Ops automation:** Admin scripts allow seeding rooms, promoting admins, and verifying email addresses without requiring UI changes, aligning with the “complete system delivery” requirement.

5. User Manual (Quick Start Guide)

5.1 Prerequisites

1. Node.js 18+ (see `package.json` engines field).
2. Firebase project with Authentication and Firestore enabled.
3. Service account JSON placed at `web/serviceAccountKey.json` or supplied via `FIREBASE_SERVICE_ACCOUNT`.
4. Environment variables defined in `web/.env` (sample file already provided).
5. Optional: Firebase Emulator Suite for local integration testing.

5.2 Installation and Local Development

```
git clone https://github.com/SawaabA/HawkSpace.git
cd web
npm install
npm run dev      # http://localhost:5173
npm run build    # production bundle in dist/
npm run preview  # serve the production bundle locally
npm run lint     # ESLint
```

If you plan to export the Markdown to PDF, install Marp globally or as a dev

LH

LAURIER CAMPUS

HawkSpace

Test Use

admin - America/Toronto

Log out

Find a Space

Request a Room

Events

My Requests

Admin

WILFRED LAURIER UNIVERSITY

Welcome Golden Hawk!

Book study halls, lecture rooms, and event spaces with the same energy you bring to campus life. We keep the process fast, transparent, and unmistakably Laurier.

Book Now

Full Events Calendar

LIT (Laurier Intro Talks) Drop-In Session

2024-09-09 5:00 PM – 7:00 PM

Lazaridis Hall 205

Meet student mentors, plan your semester and discover every space you can book on campus.

See details →

CAMPUSES & LOCATIONS

Tour Laurier spaces before you book

Need inspiration? See where Waterloo, Brantford, and Milton campuses shine, then bring that energy to HawkSpace bookings.

Visit Laurier campuses

SEARCH AVAILABLE ROOMS

Search Laurier rooms

Monday–Friday, 08:30–23:00 (America/Toronto). Pending requests show in gold so you know what's queued.

Building

Any

Min capacity

e.g. 20

Date

12/09/2025

Equipment

Projector

Whiteboard

Speakers

Mic

Hdmi

MATCHING ROOMS (15)

Arts C 120

Arts

Capacity 28 - projector, whiteboard

Arts C 223

Arts

Capacity 40 - whiteboard

Bricker Academic 201

Bricker Academic

Capacity 45 - projector, whiteboard

Bricker Academic 210

Bricker Academic

Capacity 50 - projector, speakers, assistive listening

Lazaridis Hall 101

Lazaridis Hall

Capacity 60 - projector, speakers, hdmi

Lazaridis Hall 205

Lazaridis Hall

Capacity 40 - projector, speakers

Lazaridis Hall 301

Lazaridis Hall

Capacity 90 - projector, speakers, lecture capture

Lazaridis Hall 312

Lazaridis Hall

Capacity 60 - projector, whiteboard

Library Basement 1

Library

Capacity 10 - whiteboard

Peters 101

Peters

Capacity 20 - projector

Peters 204

Peters

Capacity 32 - projector, whiteboard, podium

Research & Academic Centre 100

Research & Academic Centre

Capacity 200 - projector, speakers, stage lighting

Science Building 115

Science

Capacity 120 - projector, speakers, lab benches

Science Building 201

Science

Capacity 30 - projector, whiteboard

Science Building 310

Science

Capacity 25 - whiteboard, hdmi

Timeline — Arts C 120

Mon, Dec 8

Available

Modified

Pending

Approved

08:30 AVAILABLE	09:00 AVAILABLE	09:30 AVAILABLE	10:00 AVAILABLE
10:30 AVAILABLE	11:00 AVAILABLE	11:30 AVAILABLE	12:00 AVAILABLE
12:30 AVAILABLE	13:00 AVAILABLE	13:30 AVAILABLE	14:00 AVAILABLE
14:30 AVAILABLE	15:00 AVAILABLE	15:30 AVAILABLE	16:00 AVAILABLE
16:30 AVAILABLE	17:00 AVAILABLE	17:30 AVAILABLE	18:00 AVAILABLE
18:30 AVAILABLE	19:00 AVAILABLE	19:30 AVAILABLE	20:00 AVAILABLE
20:30 AVAILABLE	21:00 AVAILABLE	21:30 AVAILABLE	22:00 AVAILABLE
22:30 AVAILABLE			

Request this room

Figure 3: Search Availability screen with filters and slot timeline

11

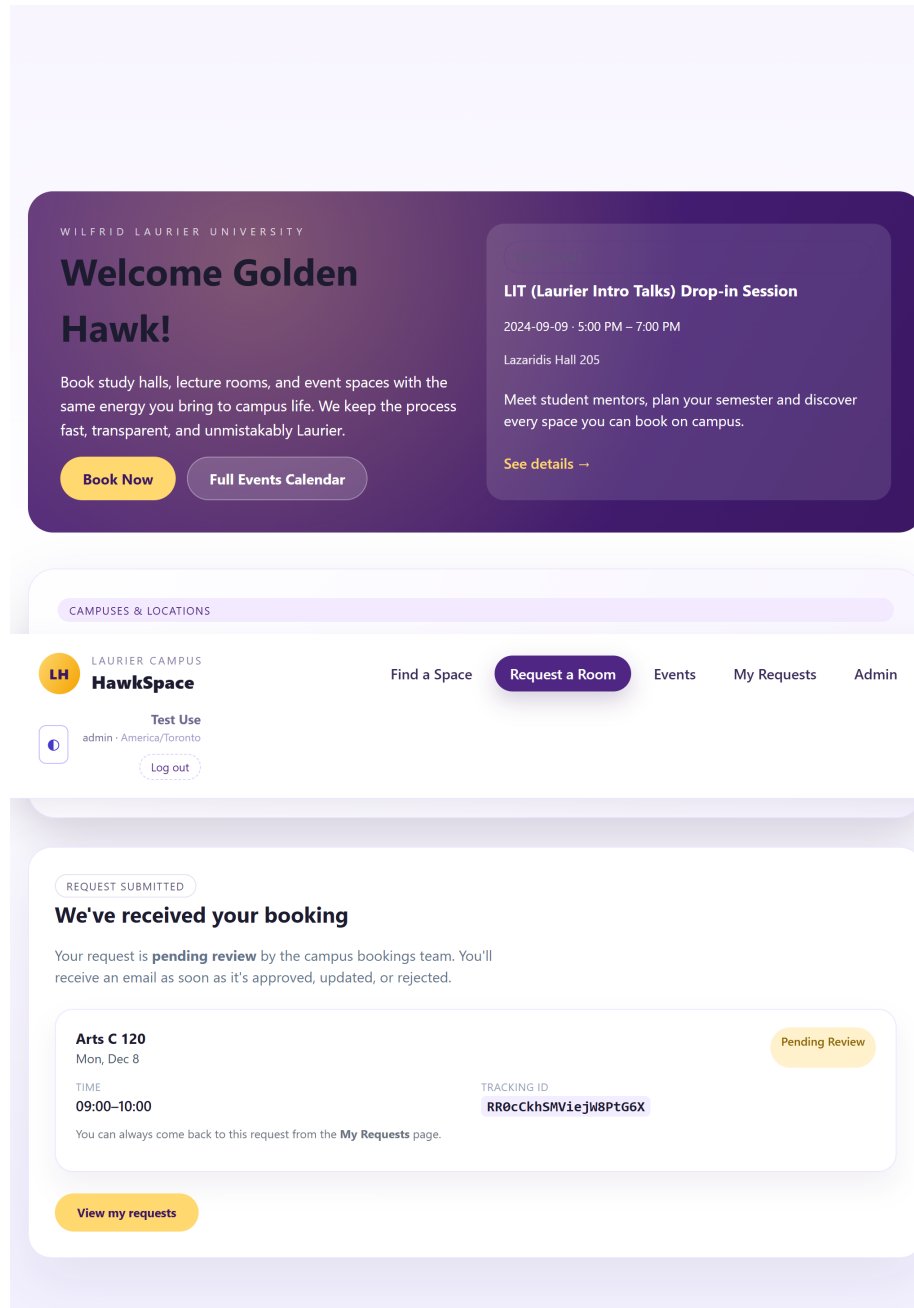


Figure 4: Request Booking success card with tracking ID

WILFRID LAURIER UNIVERSITY

Welcome Golden Hawk!

Book study halls, lecture rooms, and event spaces with the same energy you bring to campus life. We keep the process fast, transparent, and unmistakably Laurier.

Book Now
Full Events Calendar

LIT (Laurier Intro Talks) Drop-in Session

2024-09-09 - 5:00 PM – 7:00 PM

Lazaridis Hall 205

Meet student mentors, plan your semester and discover every space you can book on campus.

See details --

CAMPUSES & LOCATIONS

Tour Laurier spaces before you book

Need inspiration? See where Waterloo, Brantford, and Milton campuses shine, then bring that energy to HawkSpace bookings.

Visit Laurier campuses

REQUEST A SPACE

Request a Classroom

Maximum duration 5 hours (America/Toronto). Tell us what you need and we'll do the rest.

Room

LAURIER, CAMPUS

LH HawkSpace

Find a Space

Request a Room

Events

My Requests

Admin

Test Use

admin - America/Toronto

Log out

Notes (optional)

Single test booking for demo screenshots.

☒ Make this a recurring booking

Recurrence type

Weekly (same day each week)

Number of occurrences

4

This will create 4 bookings weekly starting from 2025-12-09

Selected time overlaps an approved or pending request.

Submit request

Timeline — Arts C 120

Mon, Dec 8

Available

Pending

Modified

Approved

08:30 AVAILABLE	09:00 PENDING	09:30 PENDING	10:00 AVAILABLE	10:30 AVAILABLE	11:00 AVAILABLE	11:30 AVAILABLE	12:00 AVAILABLE	12:30 AVAILABLE
13:00 AVAILABLE	13:30 AVAILABLE	14:00 AVAILABLE	14:30 AVAILABLE	15:00 AVAILABLE	15:30 AVAILABLE	16:00 AVAILABLE	16:30 AVAILABLE	17:00 AVAILABLE
17:30 AVAILABLE	18:00 AVAILABLE	18:30 AVAILABLE	19:00 AVAILABLE	19:30 AVAILABLE	20:00 AVAILABLE	20:30 AVAILABLE	21:00 AVAILABLE	21:30 AVAILABLE
22:00 AVAILABLE	22:30 AVAILABLE							

Figure 5: Recurring booking wizard with conflict warning and timeline

13

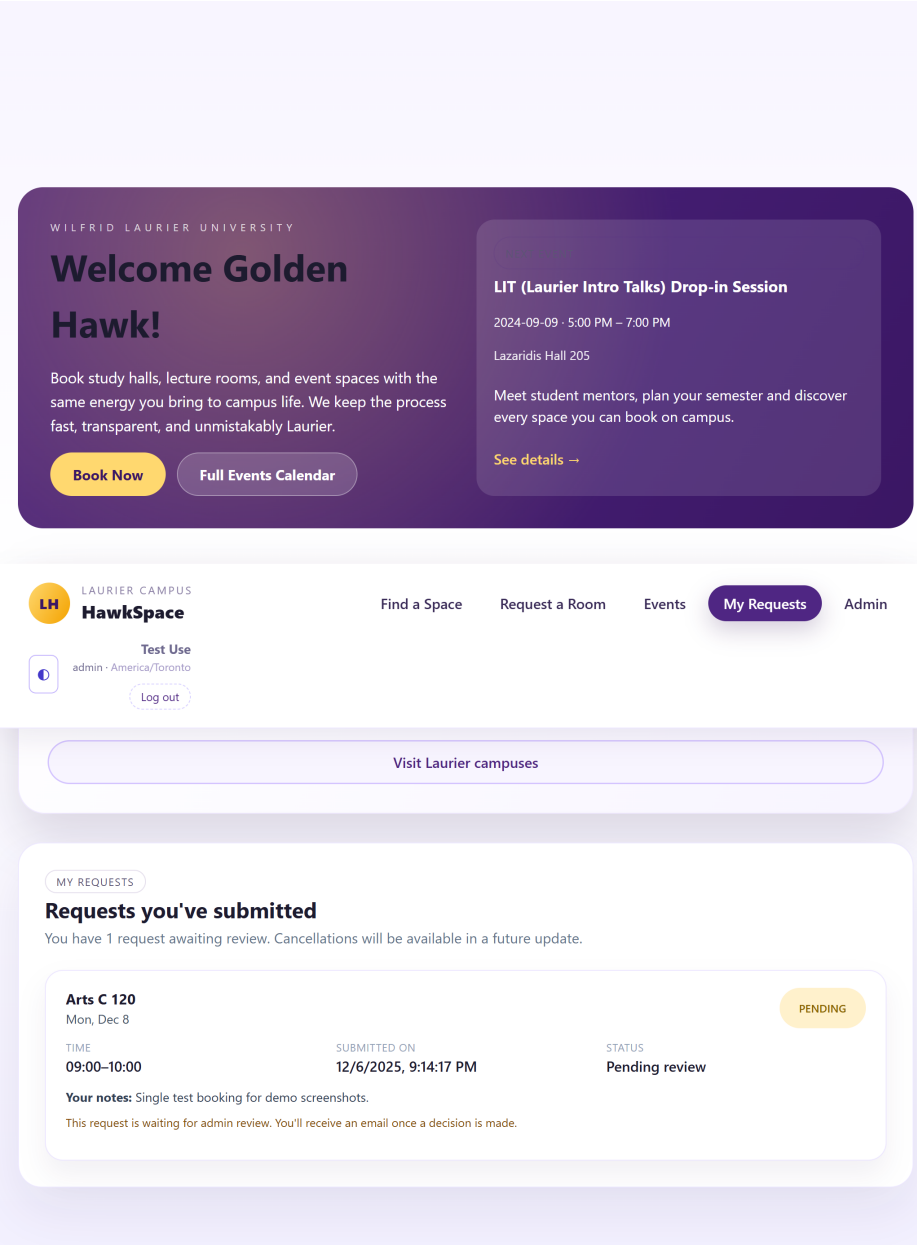


Figure 6: My Requests list showing statuses and admin notes

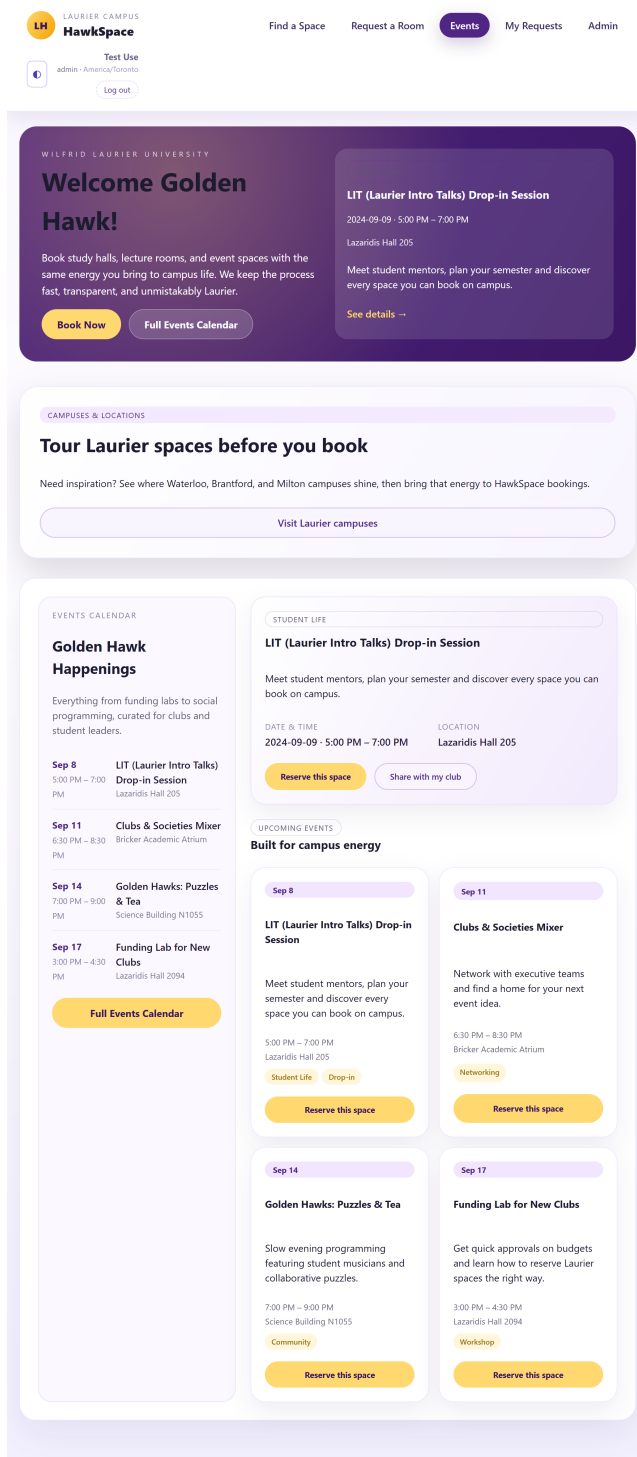


Figure 7: Events page with curated campus events

HawkSpace Admin
America/Toronto

RequestsReportsStudent view

Test Use
admin

Log out

Pending Requests

Date filter12/09/2025

Arts C 120
Mon, Dec 8
Requested by Test Use
09:00–10:00

Pending

Lazaridis Hall 205
Mon, Dec 8
Requested by Test Student
13:00–14:00

Pending

Arts C 120
Mon, Dec 8
09:00–10:00
Arts - Capacity 28

Test Use
test@mylaurier.ca

Requester notes: Single test booking for demo screenshots.

Admin note
Decision notes or edits

Date12/09/2025Start09:00End10:00

ApproveModify windowOverrideReject

Room calendar

08:30 available	09:00 pending	09:30 pending	10:00 available
10:30 available	11:00 available	11:30 available	12:00 available
12:30 available	13:00 available	13:30 available	14:00 available
14:30 available	15:00 available	15:30 available	16:00 available
16:30 available	17:00 available	17:30 available	18:00 available
18:30 available	19:00 available	19:30 available	20:00 available
20:30 available	21:00 available	21:30 available	22:00 available
22:30 available			

Figure 8: Admin Pending Requests queue with conflict calendar

16

HawkSpace Admin

America/Toronto

Requests

Reports

Student view

Test Use

admin

Log out

Pending Requests

Date filter12/09/2025

Lazaridis Hall 205

Mon, Dec 8

Requested by Test Student

13:00-14:00

Pending

Lazaridis Hall 205

Test Student

testus123student@mylaurier.ca

Mon, Dec 8

13:00-14:00

Lazaridis Hall · Capacity 40

Requester notes: Weekly Club Meeting - RECURRING TEST

Admin note

Decision notes or edits

Date12/09/2025Start13:00End14:00

Approve

Modify window

Override

Reject

Override completed ✓

Room calendar

08:30 available	09:00 available	09:30 available	10:00 available
10:30 available	11:00 available	11:30 available	12:00 available
12:30 available	13:00 pending	13:30 pending	14:00 available
14:30 available	15:00 available	15:30 available	16:00 available
16:30 available	17:00 available	17:30 available	18:00 available
18:30 available	19:00 available	19:30 available	20:00 available
20:30 available	21:00 available	21:30 available	22:00 available
22:30 available			

Figure 9: Admin override action with override completed history and room calendar

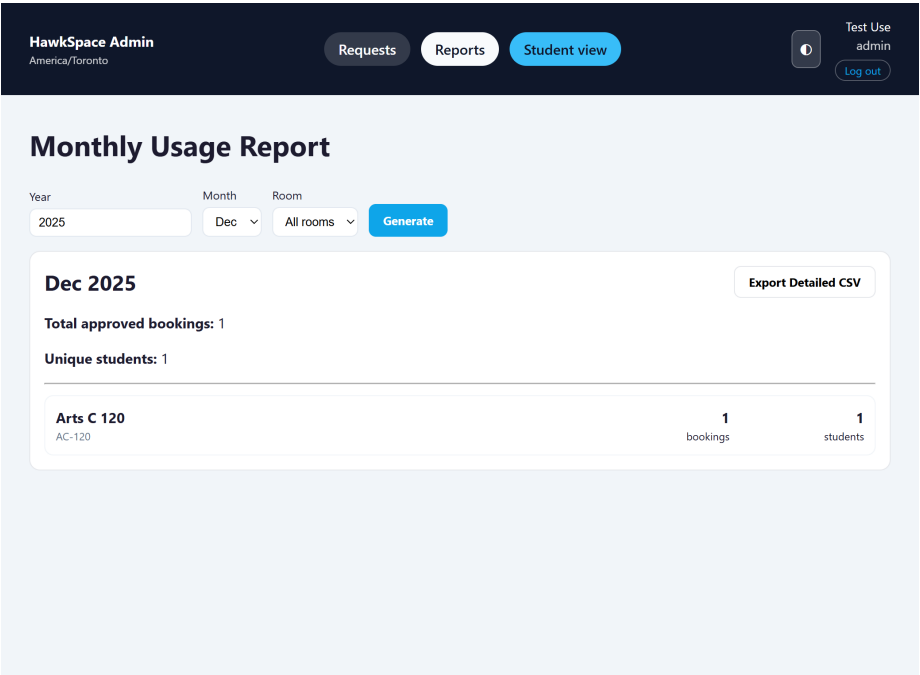


Figure 10: Admin Monthly Usage Report with CSV export button

Laurier Campus

HawkSpace

Test Use

Address: Waterloo Campus

Log Out

Request a Room

Events

My Requests

Admin

WILFRID LAURIER UNIVERSITY

Welcome Golden Hawk!

Book study halls, lecture rooms, and event space with the same energy you bring to campus life. We keep the process fast, transparent, and unmistakably Laurier.

Book Now

Fall Events Calendar

LIT (Laurier Intro Talks) Drop-in Session

2024-09-09 5:00 PM – 7:00 PM

Lazaridis Hall 205

Meet student mentors, plan your semester and discover every space you can book on campus.

[See details...](#)

CAMPUSES & LOCATIONS

Tour Laurier spaces before you book

Need inspiration? See where Waterloo, Brantford, and Milton campuses shine, then bring that energy to HawkSpace bookings.

Visit Laurier campuses

SEARCH AVAILABLE ROOMS

Search Laurier rooms

Monday–Friday, 08:30–23:00 (America/Toronto). Pending requests show in gold so you know what's queued.

Building

Min capacity

Date

Any

0-0 / 20

12/09/2025

Equipment

Projector

Whiteboard

Speakers

Mic

Other

MATCHING ROOMS (13)

Arts C 120

Arts

Capacity 20 · projector, whiteboard

Arts C 223

Arts

Capacity 40 · whiteboard

Bricker Academic 201

Bricker Academic

Capacity 45 · projector, whiteboard

Bricker Academic 210

Bricker Academic

Capacity 50 · projector, speakers, assistive listening

Lazaridis Hall 101

Lazaridis Hall

Capacity 60 · projector, speakers, hdmi

Lazaridis Hall 205

Lazaridis Hall

Capacity 40 · projector, speakers

Lazaridis Hall 301

Lazaridis Hall

Capacity 50 · projector, speakers, lecture capture

Lazaridis Hall 312

Lazaridis Hall

Capacity 60 · projector, whiteboard

Library Basement 1

Library

Capacity 10 · whiteboard

Peters 101

Peters

Capacity 20 · projector

Peters 204

Peters

Capacity 32 · projector, whiteboard, podium

Research & Academic Centre 100

Research & Academic Centre

Capacity 200 · projector, speakers, stage lighting

Science Building 115

Science

Capacity 120 · projector, speakers, lab benches

Science Building 301

Science

Capacity 30 · projector, whiteboard

Science Building 310

Science

Capacity 25 · whiteboard, hdmi

Timeline — Arts C 120

Mon, Dec 8

Available

Pending

Modified

Approved

08:30 AVAILABLE

09:00 APPROVED

09:30 APPROVED

10:00 AVAILABLE

10:30 AVAILABLE

11:00 AVAILABLE

11:30 AVAILABLE

12:00 AVAILABLE

12:30 AVAILABLE

13:00 AVAILABLE

13:30 AVAILABLE

14:00 AVAILABLE

14:30 AVAILABLE

15:00 AVAILABLE

15:30 AVAILABLE

16:00 AVAILABLE

16:30 AVAILABLE

17:00 AVAILABLE

17:30 AVAILABLE

18:00 AVAILABLE

18:30 AVAILABLE

19:00 AVAILABLE

19:30 AVAILABLE

20:00 AVAILABLE

20:30 AVAILABLE

21:00 AVAILABLE

21:30 AVAILABLE

22:00 AVAILABLE

22:30 AVAILABLE

Request this room

Figure 11: High-contrast accessibility mode on the main search page

19

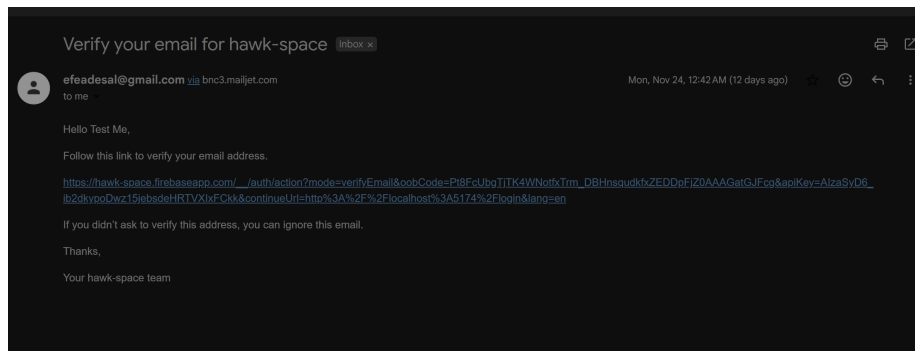


Figure 12: Email verification message for HawkSpace account

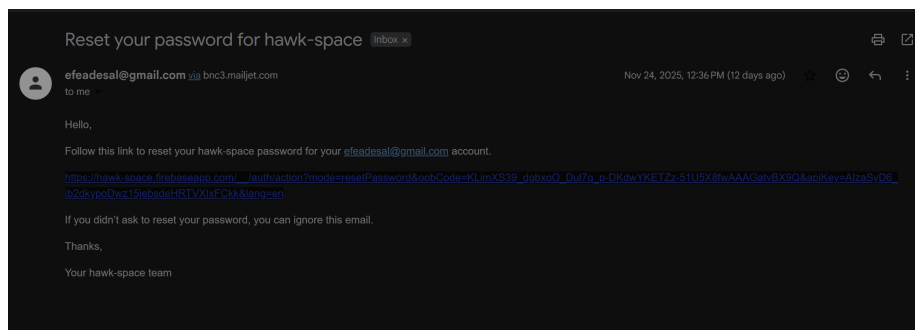


Figure 13: Reset password email for HawkSpace account

dependency:

```
npx @marp-team/marp-cli documents/HawkSpace_Final_Submission.md --allow-local-files --pdf
```

5.3 Firebase Setup and Admin Operations

1. **Seed rooms:** `npm run seed-rooms` populates Firestore with rooms from `src/data/rooms.js`.
2. **Create admin:** `npm run create-admin admin@mylaurier.ca Admin123! "Campus Admin"` (email must match an allowed domain).
3. **Verify users:** If email verification lags (school-managed email accounts can delay messages for hours), run `npm run verify-email user@mylaurier.ca` or start `npm run verify-server` and POST `{"email":"..."}` to `http://localhost:3001/verify-email`.
4. **Persistence toggle:** The app reads the `auth_persistence` key from `localStorage` to determine local vs. session persistence.
5. **Recurring operations:** Use `cancelRecurringSeries` from `MyRequests` to test multi-booking cancellation.

5.4 Test Credentials & Access Levels

- **Student / Club:** Create through the Signup page or Firebase console. Example: `clubexec.demo@mylaurier.ca / Demo123!` (verify via script).
- **Admin:** Result of `npm run create-admin`. Example: `admin@mylaurier.ca / Admin123!` redirected to `/admin/requests`.
- **Alternate emails:** Due to spam filtering on institutional accounts, students can temporarily use Gmail for testing, as documented by the Product Owner.

5.5 Feature Walkthrough (cross-reference story IDs)

1. **Search for rooms (UI-1):** Set filters, pick a date, review slot colors (available, pending, modified, approved).
2. **Submit a booking (UI-2):** Fill out room, date, start/end slot, notes, and optionally enable recurring bookings. Success screen displays a tracking ID.
3. **Manage requests (UI-4/UI-5):** View statuses, admin notes, and cancel entire recurring series. Single-booking cancellation is an admin-only action for now.
4. **Admin review (ADM-1/ADM-3):** Approve, modify, override, or reject requests. Override modal requires reason and priority level; actions write to history.
5. **Reports (REP-1):** Generate monthly statistics by month/year/room, then export CSV for Excel (BOM included).
6. **Dashboard overview (DASH-1):** Student hero surfaces next events and CTA buttons; events page deep-dives into programming.

7. **Notifications (NOTIF-1):** Design supports confirmation emails, but due to spam delays the feature is currently disabled pending a reliable mail provider. Mention this in demos to set expectations.

6. Summary of Testing (Methodology, Evidence, Results)

A. Methodology

- **Manual feature tests:** Chrome and Edge coverage of each student and admin flow, repeated at every sprint review.
- **Integration tests with Firebase Emulator:** Validated Firestore transactions, slot conflicts, override handling, and recurring series creation/cancellation.
- **Script verification:** Ops scripts run against service accounts to ensure seeding, admin creation, and email verification succeed.
- **UX/accessibility checks:** High-contrast toggle, keyboard navigation, and color contrast verified during UX-1/UX-2 acceptance.
- **Evidence sources:** Manual demos, emulator sessions, ops-script dry runs, and accessibility walkthroughs conducted during each sprint review.

B. Results Table

Story				
ID	Test Performed	Expected Result	Actual Result	Pass/Fail
UI-1	Filter Science rooms to capacity ≥ 25 with projector and inspect timeline	Only SB-201 and SB-310 listed; slot colors accurate	Filters reduced to two rooms and timeline matched Firestore data	Pass
UI-2	Submit single booking for LH-205 10:00-12:00 with notes	Request stored, conflict prevented, success card shown	Booking saved with pending status and displayed tracking ID	Pass
UI-3	Cancel booking from admin override	Conflicting booking freed, history recorded	Override modal cancelled approved booking and logged action	Pass
UI-5	Create weekly recurring booking (4 weeks) then cancel series	Four requests created; Cancel Series removes them and frees slots	Series stored, cancellation set status to “cancelled” and cleared slots	Pass
ADMA1	Approve pending request with admin note	Status becomes approved, admin note saved, timeline updates	Request updated, My Requests pill turned green, day slots locked	Pass

Story				
ID	Test Performed	Expected Result	Actual Result	Pass/Fail
REP-1	Generate September report for all rooms and export CSV	Summary metrics and CSV with BOM for Excel	Report aggregated counts and CSV opened correctly in Excel	Pass
UI-6	Toggle high-contrast mode in student and admin shells	Body class toggles, colors adjust, preference persists	Toggle persisted via localStorage and updated UI instantly	Pass

C. Known Issues and Limitations

- Outbound notifications are still disabled until a reliable mail service is integrated.
- Student self-service cancellation for single bookings is not implemented (admin intervention required).
- Weekly quota policies (ADM-2 follow-up) are limited to per-request max duration.
- Timezone logic assumes America/Toronto; multi-campus rollouts require per-room timezone metadata.
- Some legacy strings contained encoding artifacts (“A-”); these were cleaned in this document but should be scrubbed from UI copy as well.

7. Ethical Dimensions Report (Privacy, Accessibility, Fairness)

Privacy & Security Considerations

- **Data minimization:** Only names, email addresses, club notes, and booking metadata are stored. No passwords or sensitive club documents are persisted.
- **Access controls:** AuthContext enforces domain checks, Firestore security rules (to be finalized) should restrict operations per role, and admin features require verified admin claims.
- **Audit trails:** Every change writes to booking history, giving admins accountability and discouraging misuse (e.g., mass fake bookings).
- **Risks:** Unauthorized access could expose club plans, and role misconfigurations could let students see other clubs’ requests. Mitigation comes from strict security rules and script-driven admin setup.
- **Future improvements:** Stronger encryption at rest (server-side), automated intrusion detection, explicit privacy notices, and purging of expired bookings after retention windows.

Because the room-booking system handles student and administrator information, it must protect privacy and security. The system stores only the minimal identifiers (name, email, club role, booking metadata) and relies on school-managed

accounts for an additional layer of protection. Role-based authentication prevents students from seeing other clubs' bookings, and every request is logged so administrators can trace misuse. Booking requests remain reviewable, reducing the risk of fake submissions or disinformation. Future improvements include stronger encryption, automated intrusion detection, and formal privacy documentation.

Accessibility & Fairness

- **Accessibility work to date:** Clear text labels, descriptive buttons, keyboard-friendly forms, high-contrast mode, and consistent layout across sections. These were validated during UX-1/UX-2 acceptance and are part of Section 6 tests.
- **Remaining gaps:** Full mobile optimization (UX-4) and complete screen-reader auditing are pending; future work should add focus outlines to timelines and ensure ARIA relationships for grouped cards.
- **Fairness:** Requests are timestamped first-come-first-served, override reasons are mandatory, and admin dashboards present identical data for each request to avoid bias.
- **Future fairness enhancements:** Automatic weekly quota checks, analytics that highlight underserved clubs, and anonymized review modes would limit unconscious bias.

Ensuring accessibility is a key responsibility for a campus-wide tool, so the system follows guidelines such as clear text labels, readable fonts, consistent navigation, and strong color contrast. Pages are structured for screen readers, and forms use predictable layouts. Fairness is maintained by timestamping booking requests (first-come-first-served), displaying consistent room information to all clubs, and presenting admin dashboards uniformly to reduce bias. Future enhancements include automated conflict detection, standardized approval rules, and equitable booking limits.

Ethical Reflection

The project respects privacy by limiting data collection, requires authentication to prevent random submissions, and provides clear audit logs for admin actions. However, there is room to improve security (session timeouts, encryption), accessibility (full keyboard navigation in timelines), and fairness (automated checks on room distribution). Implementing those improvements and documenting practices will increase trust among students and staff.

The project delivers a functioning booking workflow—room catalogue, availability calendar, booking form, approval system, events calendar, and booking history—supported by authentication, role-based access, and Firebase security. To improve further, add session timeout management, deepen accessibility for screen readers and keyboard users, and provide analytics that help administrators make consistent decisions (e.g., display booking frequency per club). Enabling students to edit requests would reduce admin workload and increase trust.

8. Final Reflection

- **Completion:** The delivered system meets MVP goals—students can browse rooms, check availability, submit bookings (single or recurring), follow statuses, and view campus events; admins can approve, modify, override, and report on room usage.
- **Biggest technical and collaboration challenges:**
 1. **Collaboration tooling:** Mixed experience with GitHub plus Mac vs. Windows environments caused merge friction. Muqadas coordinated workflows, but the team still spent extra time resolving conflicts.
 2. **Email deliverability:** University-managed accounts introduced multi-hour delays and spam filtering, forcing the team to allow alternative email addresses and build verification scripts.
 3. **Complex transactions:** Designing slot-based Firestore transactions for overrides and recurring series required careful reasoning to avoid conflicts and partial failures.
- **What to improve with more time:**
 1. Allow students to edit or cancel single requests without admin help.
 2. Finish responsive/mobile optimization and deeper accessibility checks (screen readers, keyboard focus).
 3. Automate fairness analytics (e.g., show booking counts per club during admin review).
 4. Harden notification workflows with a trusted email provider, session timeouts, and monitoring.
 5. Integrate institutional SSO once access is granted.
- **Next steps:** Finalize Firestore security rules, add CI/CD with automated tests (React Testing Library + Firebase emulator), complete deferred backlog stories (UX-4, EMAIL-2, ADMIN-4, INT-1), and embed the system architecture diagram plus requested screenshots before exporting this Markdown to PDF.

The project successfully delivered a working room-booking web application that meets its main functional goals. Students can browse rooms, check availability, submit booking requests, and view upcoming events, while administrators can review and approve requests efficiently. The biggest technical challenges involved collaboration (mixed GitHub experience and Mac/Windows differences) and email notification deliverability (school-managed accounts often delayed or filtered confirmations). Allowing students to enter alternate emails mitigated the latter issue. If given more time, the team would enable students to edit requests, enhance accessibility for users with disabilities, implement fairness analytics for administrators, and optimize the notification pipeline to reduce delays and improve reliability.

Prepared for submission as a single comprehensive document. Convert this Markdown to PDF (e.g., `npa @marp-team/marp-cli documents/HawkSpace_Final_Submission.md --allow-local-files -pdf`), embed the architecture image and UI screen-

shots, and include any supporting images directly in the final export.